Searching

Apa Itu Searching?

Searching atau yang bahasa indonesia nya adalah Pencarian, merupakan sebuah proses dasar untuk menemukan posisi, suatu elemen, atau item tertentu dalam kumpulan data. Kumpulan data yang dimaksut memiliki berbagai macam jenis, antara lain seperti array, list, tree, dan lainnya. Tujuan dilakukannya Searching adalah untuk menemukan elemen yang diinginkan terdapat didalam data, jika ditemukan, maka lokasi dari elemen tersebut akan diidentifikasi.

Karakteristik dari Searching adalah

- Elemen/Target

  Saat terjadinya proses searching, akan selalu ada elemen atau target spesifik yang ingin ditemukan dalam kumpulan data.
- Ruang Pencarian

  Tempat mencari target yang berada pada kumpulan data, kumpulan data dapat sangat sedikit ataupun sangat banyak. Selain itu, hal ini sangat bergantung pada bagaimana data tersebut diorganisasikan, jika kumpulan datanya tidak beraturan maka proses searching dapat menjadi lebih lama, sebaliknya jika kumpulan datanya beraturan maka proses searching dapat menjadi lebih cepat.
- Kompleksitas

  Tingkat kerumitan suatu metode searching. Selain itu, kompleksitas sangat bergantung pada jenis data dan metode yang digunakan, biasanya mempertimbangkan berapa banyak waktu dan ruang yang dibutuhkan.
- Deterministik vs Non-deterministik

  Beberapa algoritma pencarian, seperti pencarian biner, bersifat deterministik, artinya mereka mengikuti pendekatan yang jelas dan sistematis. Yang lain, seperti pencarian linier, bersifat non-deterministik, karena dalam kasus terburuk, mereka mungkin perlu memeriksa seluruh ruang pencarian.

Algoritma Searching

Algoritma Searching adalah sebuah algoritma yang dirancang untuk memeriksa posisi suatu elemen atau mengambil suatu elemen dari struktur data mana pun tempat elemen tersebut disimpan.

Beberapa Algoritma Searching yang akan dibahas pada modul ini yaitu:

Fibonacci Search

Fibonacci Search mencari nilai di array terurut dengan cara membagi array menggunakan bilangan Fibonacci, bukan tepat di tengah seperti Binary Search. Pembagian menggunakan Fibonacci membuat algoritma ini hanya butuh operasi penjumlahan dan pengurangan. Tidak ada pembagian (/) sama sekali.

Barisan Fibonacci

F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5,
F(6)=8, F(7)=13, F(8)=21, F(9)=34, ...

Rumus: F(k) = F(k-1) + F(k-2)

array berukuran n dibagi di posisi F(k-2) (bukan di n/2)

Cara Kerja

1. Cari bilangan Fibonacci terkecil yang nilainya >= n (ukuran array).
2. Simpan tiga bilangan Fibonacci berturut: fib2 (F(k-2)), fib1 (F(k-1)), fib (F(k)).
3. Gunakan offset = -1 sebagai batas kiri elemen yang sudah dibuang.
4. Tentukan indeks i = offset + fib2, lalu bandingkan arr[i] dengan target:
   - arr[i] == target → ketemu, kembalikan i
   - arr[i] < target → buang bagian kiri, geser offset ke i, turunkan Fibonacci
   -arr[i] > target → buang bagian kanan, perkecil Fibonacci
5. Ulangi sampai fib <= 1 atau ditemukan.

Contoh Manual

Cari 85 di array [10, 22, 35, 40, 45, 50, 80, 82, 85, 90, 100]

n = 11. Fibonacci naik sampai >= 11: fib=13, fib1=8, fib2=5

offset = -1

Langkah 1:

i = -1+5 = 4 → arr[4]=45 < 85

buang kiri, offset=4, fib=8, fib1=5, fib2=

Langkah 2:

i = 4+3 = 7 → arr[7]=82 < 85

buang kiri, offset=7, fib=5, fib1=3, fib2=2

Langkah 3:

i = 7+2 = 9 → arr[9]=90 > 85

buang kanan, fib=2, fib1=1, fib2=1

Langkah 4:

i = 7+1 = 8 → arr[8]=85 == 85

KETEMU di indeks 8

Kompleksitas

Kasus	Waktu	Keterangan
Best Case	O(1)	Langsung ketemu di pengecekan pertama
Worst Case	O(log n)	Pencarian sampai range paling kecil
Space	O(1)	Hanya 3 variabel Fibonacci + offset

Source Code Fibonacci Search

Berikut adalah implementasi Fibonacii Search pada Single Linked List.

```
#include <iostream>
using namespace std;

// Fungsi Fibonacci Search
// arr  : pointer ke array (bisa juga ditulis int arr[])
// n    : jumlah elemen
// target : nilai yang dicari
// return  : indeks jika ketemu, -1 jika tidak


int fibonacciSearch(int *arr, int n, int target) {

    // Tiga variabel Fibonacci berturut-turut
    int fib2 = 0;   // F(k-2)
    int fib1 = 1;   // F(k-1)
    int fib  = 1;   // F(k)  = fib1 + fib2

    // Naikkan Fibonacci sampai >= n
    while (fib < n) {
        fib2 = fib1;
        fib1 = fib;
        fib  = fib1 + fib2;
    }

    int offset = -1;   // batas kiri yang sudah dieliminasi

    while (fib > 1) {
        // Tentukan indeks yang akan diperiksa
        int i = offset + fib2;
        if (i >= n) i = n - 1;  // jangan lewati batas array

        if (arr[i] == target)
            return i;                // KETEMU
        else if (arr[i] < target) {
            // Cari di sebelah KANAN → geser offset
            fib  = fib1;
```

```
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        } else {
            // Cari di sebelah KIRI → perkecil range
            fib  = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib  - fib1;
        }
    }

    // Periksa satu elemen tersisa
    if (fib1 == 1 && arr[offset + 1] == target)
        return offset + 1;

    return -1;   // TIDAK KETEMU
}

int main() {
    int arr[] = {10, 22, 35, 40, 45, 50, 80, 82, 85, 90, 100};
    int n     = 11;
    int target;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    cout << "Masukkan nilai yang dicari: ";
    cin  >> target;

    int hasil = fibonacciSearch(arr, n, target);

    if (hasil != -1)
        cout << "Ditemukan di indeks " << hasil << endl;
    else
        cout << "Tidak ditemukan." << endl;

    return 0;
}
```

Jump Search

Jump Search bekerja pada array terurut dengan cara melompat maju sebesar blok n^2, baru kemudian melakukan pencarian linear mundur di dalam blok yang tepat. Idenya: daripada memeriksa satu per satu dari awal (terlalu lambat), kita loncat n^2 langkah sekaligus, lalu mundur sedikit untuk mencari dengan tepat.

Mengapa lompatan optimal = n^2 ?

Misalkan ukuran array = n, ukuran lompatan = k.

Jumlah lompatan yang dibutuhkan = n / k

Jumlah langkah linear setelah lompat = k

Total operasi = n/k + k

Minimumkan n/k + k → turunkan terhadap k:

d/dk (n/k + k) = $-n/k^2 + 1 = 0$ → $k^2 = n$ → $k = n^2$

Jadi lompatan n^2 meminimalkan total operasi → $O(n^2)$.

Cara Kerja

1. Hitung step = n^2(dibulatkan ke bawah).
2. FASE LOMPAT: Selama arr[step-1] < target dan step < n, lompat maju: prev = step, step += n^2.
3. FASE LINEAR: Cari satu per satu dari indeks prev sampai step (atau n jika step > n).
4. Jika arr[prev] == target → ketemu. Jika prev sampai ujung blok → tidak ketemu.

Contoh Manual

Cari 25 di array [1, 3, 5, 7, 9, 11, 19, 25, 30, 36, 50]

n = 11, step = √11 ≈ 3, prev = 0

=== FASE LOMPAT ===

Lompat 1: arr[3-1] = arr[2] = 5 < 25 → prev = 3, step = 6

Lompat 2: arr[6-1] = arr[5] = 11 < 25 → prev = 6, step = 9

Lompat 3: arr[9-1] = arr[8] = 30 >= 25 → BERHENTI

=== FASE LINEAR (prev=6 sampai step=9) ===

arr[6] = 19 < 25 → prev = 7

arr[7] = 25 == 25 → KETEMU di indeks 7

Kompleksitas

Kasus	Waktu	Keterangan
Best Case	O(1)	Target tepat di posisi pertama yang dicek
Worst Case	O(√n)	Target di akhir blok terakhir
Space	O(1)	Hanya variabel step, prev, dan batas

Source Code Jump Search

Berikut adalah implementasi Jump Search pada Single Linked List menggunakan bahasa pemrograman C++

```
#include <iostream>
#include <cmath>    // untuk sqrt()
using namespace std;

// Fungsi Jump Search
// arr  : pointer ke array terurut
// n    : jumlah elemen
// target : nilai yang dicari
int jumpSearch(int *arr, int n, int target) {

    int step = (int)sqrt((double)n);  // ukuran satu lompatan
    int prev = 0;                     // awal blok saat ini
```

```
    // === FASE 1: LOMPAT antar blok ===
    // Lompat selama ujung blok masih < target
    while (arr[step - 1] < target) {
        prev = step;
        step += (int)sqrt((double)n);

        if (prev >= n)
            return -1;   // sudah melewati semua elemen
    }

    // === FASE 2: LINEAR SEARCH dalam blok ===
    // Cari satu per satu dari prev sampai ujung blok
    int batas = step;
    if (batas > n) batas = n;   // jangan lewati akhir array

    while (prev < batas) {
        if (arr[prev] == target)
            return prev;         // KETEMU
        prev++;
    }

    return -1;   // TIDAK KETEMU
}

int main() {
    int arr[] = {1, 3, 5, 7, 9, 11, 19, 25, 30, 36, 50};
    int n     = 11;
    int target;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    cout << "Masukkan nilai yang dicari: ";
    cin  >> target;

    int hasil = jumpSearch(arr, n, target);

    if (hasil != -1)
        cout << "Ditemukan di indeks " << hasil << endl;
```

```
    else
        cout << "Tidak ditemukan." << endl;

    return 0;
}
```

Boyer-Moore Search

Boyer-Moore adalah algoritma pencarian pola (pattern matching) — ia mencari apakah sebuah string pendek (pattern) ada di dalam string panjang (text). Yang unik, perbandingannya dilakukan dari KANAN ke KIRI, lalu bila ada ketidakcocokan, pattern digeser ke depan sejauh mungkin menggunakan tabel Bad Character.

Dua Konsep Kunci Boyer-Moore

1. Perbandingan Kanan ke Kiri:
   Pattern dicocokkan mulai karakter TERAKHIR, bukan pertama.
   Ini memungkinkan lompatan besar ketika ada ketidakcocokan.
2. Bad Character Heuristic:
   Ketika terjadi mismatch di text[s+j], lihat karakter text[s+j] di tabel.
   Geser pattern supaya kemunculan terakhir karakter itu di pattern sejajar.
   Jika karakter itu tidak ada di pattern sama sekali, geser pattern penuh.

Cara Kerja

1. PREPROCESSING — Buat tabel badChar[256]. Untuk setiap karakter ASCII, simpan indeks kemunculan terakhirnya di pattern. Kalau tidak ada, isi -1.
2. Mulai dengan s = 0 (posisi pattern sejajar awal text).
3. Bandingkan pattern dari kanan ke kiri (j dari m-1 turun ke 0).
4. Jika j < 0 (semua cocok): pattern DITEMUKAN di posisi s. Hitung shift untuk mencari kemunculan berikutnya.
5. Jika ada mismatch di posisi j: geser = j - badChar[text[s+j]]. Jika geser < 1, pakai 1.
6. s += geser. Ulangi dari langkah 3.

Contoh Tabel Bad Character Pattern: "ABCAB" (m=5)

Buat tabel (hanya karakter yang relevan ditampilkan):

1. i=0: pattern[0] = 'A' → badChar['A'] = 0
2. i=1: pattern[1] = 'B' → badChar['B'] = 1
3. i=2: pattern[2] = 'C' → badChar['C'] = 2
4. i=3: pattern[3] = 'A' → badChar['A'] = 3 (timpa nilai lama)
5. i=4: pattern[4] = 'B' → badChar['B'] = 4 (timpa nilai lama)

Hasil akhir: badChar['A'] = 3, badChar['B'] = 4, badChar['C'] = 2

Karakter lain (misal 'D', 'Z', dll) = -1

Kompleksitas

Kasus	Waktu	Keterangan
Best Case	O(n/m)	Pattern tidak ada di alfabet teks, skip banyak
Average Case	O(n/m)	Praktis sangat cepat untuk teks natural
Worst Case	O(n·m)	Teks/pattern berulang, misal 'AAAA' cari 'AAA'
Preprocessing	O(m+256)	Membangun tabel bad character
Space	O(256)	Array badChar ukuran 256 (konstan)

Source Code Boyer-Moore Search

Berikut adalah implementasi Boyer-Moore Search pada Single Linked List menggunakan bahasa pemrograman C++

```
#include <iostream>
#include <string>
using namespace std;

// Ukuran alfabet (semua karakter ASCII)
#define ALFABET 256

// -------------------------------------------------------
// Buat tabel Bad Character
// Isi tabel: indeks terakhir tiap karakter dalam pattern
// Karakter yang tidak ada di pattern → bernilai -1
// -------------------------------------------------------
void buatTabelBadChar(string &pattern, int m, int tabel[]) {
    // Set semua ke -1 dulu
    for (int i = 0; i < ALFABET; i++)
        tabel[i] = -1;

    // Isi indeks terakhir tiap karakter pattern
    for (int i = 0; i < m; i++)
        tabel[(int)pattern[i]] = i;
}

// -------------------------------------------------------
// Fungsi Boyer-Moore Search
// text    : teks yang dicari
// pattern : pola yang ingin ditemukan
// -------------------------------------------------------
void boyerMoore(string &text, string &pattern) {
    int n = (int)text.size();
    int m = (int)pattern.size();

    // Buat tabel bad character
    int badChar[ALFABET];
    buatTabelBadChar(pattern, m, badChar);

    int s = 0;       // posisi geser pattern terhadap text
    bool ketemu = false;

    while (s <= n - m) {
        int j = m - 1;   // mulai dari karakter PALING KANAN pattern

        // Cocokkan dari kanan ke kiri
```

```
        while (j >= 0 && pattern[j] == text[s + j])
            j--;

        if (j < 0) {
            // Semua karakter cocok → KETEMU
            cout << "Pattern ditemukan di posisi: " << s << endl;
            ketemu = true;

            // Hitung geser berikutnya
            if (s + m < n)
                s += m - badChar[(int)text[s + m]];
            else
                s++;
        } else {
            // Ada ketidakcocokan → pakai bad character rule
            int geser = j - badChar[(int)text[s + j]];
            if (geser < 1) geser = 1;   // minimal geser 1
            s += geser;
        }
    }

    if (!ketemu)
        cout << "Pattern tidak ditemukan." << endl;
}

int main() {
    string text, pattern;

    cout << "Masukkan teks    : ";
    getline(cin, text);

    cout << "Masukkan pattern : ";
    getline(cin, pattern);
```

Perbandingan Ketiga Algoritma

Aspek	Fibonacci Search	Jump Search	Boyer-Moore
Tipe data	Array integer terurut	Array integer terurut	String / teks
Perlu urut?	Ya	Ya	Tidak
Worst case	O(log n)	$O(n^2)$	O(n·m)
Average case	O(log n)	$O(n^2)$	O(n/m)
Space	O(1)	O(1)	O(256) = O(1)
Operasi	+ dan − saja	+ dan sqrt()	Array ASCII 256
Implementasi	Sedang	Mudah	Sedang

Panduan Memilih Algoritma

Punya array angka terurut dan ingin cepat seperti Binary Search?

Fibonacci Search (tidak pakai pembagian, cocok untuk hardware terbatas)

Punya array angka terurut dan ingin kode paling simpel?

Jump Search (kode pendek, mudah dipahami, performa $O(n^2)$)

Ingin mencari kata / pola di dalam teks panjang?

Boyer-Moore (sangat cepat di praktik, standar industri)